

DD2440 Advanced Algorithms
Fall 2026
Lecture 3: Fixed-Parameter Algorithms

Ioana Bercea

2026-09-16

Agenda

Vertex Cover

Parameterized Complexity

Kernelization

Closest String

Next week

- ▶ Group assignment 1 released on Monday, due on Friday
- ▶ model solutions/hints released on Wednesday
- ▶ register groups on Canvas by the end of the week

- ▶ Exercise sessions on Tuesday and Thursday (and on Zoom)
- ▶ big chance to practice proofs

Agenda

Vertex Cover

Parameterized Complexity

Kernelization

Closest String

Dealing with intractability (recap)

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis
- ▶ Approximation algorithms
- ▶ Parameterized algorithms

← *TSP project*

← *last week*

Dealing with intractability (recap)

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms

- ▶ Heuristics

← *TSP project*

- ▶ Average-case / smoothed analysis

- ▶ Approximation algorithms

← *last week*

- ▶ Parameterized algorithms

← *today*

Vertex Cover (recap)

A vertex cover in a graph G is a set of vertices that covers (“touches”) every edge of G .

Goal: find a vertex cover of minimum size.

```
1 function VertexCoverApprox( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

This algorithm has approximation ratio 2
(finds vertex cover at most 2x larger than Opt)

Running time: can be implemented in linear time $O(|V| + |E|)$.

What if we just want to know if there is a vertex cover of size k for some *small constant* k ?

What if we just want to know if there is a vertex cover of size k for some *small constant* k ?

Discuss: what is the best running time you can get if $k = 10$?

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

What is the time complexity of this algorithm?

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

What is the time complexity of this algorithm?

- There are $\binom{n}{k}$ subsets to try.

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

What is the time complexity of this algorithm?

- ▶ There are $\binom{n}{k}$ subsets to try.
- ▶ Each subset can be tested in $O(m)$ time.

Remark: m is number of edges. For Vertex Cover we can assume that $m \geq n$

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

What is the time complexity of this algorithm?

- ▶ There are $\binom{n}{k}$ subsets to try.
- ▶ Each subset can be tested in $O(m)$ time.

Remark: m is number of edges. For Vertex Cover we can assume that $m \geq n$

- ▶ $\Rightarrow$ total time $O\left(\binom{n}{k}m\right)$

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

What is the time complexity of this algorithm?

- ▶ There are $\binom{n}{k}$ subsets to try.
- ▶ Each subset can be tested in $O(m)$ time.

Remark: m is number of edges. For Vertex Cover we can assume that $m \geq n$

- ▶ $\Rightarrow$ total time $O\left(\binom{n}{k}m\right)$

Test yourself: how large is $\binom{n}{k}$?

(i) $\approx n \cdot k$ (ii) $\approx n^k$ (iii) $\approx k^n$ (iv) $\approx 2^n$ (v) $\approx 2^k n$

Solving Vertex Cover for small k

Naive algorithm:

```
1 function VertexCoverNaive( $G, k$ ):  
2   foreach subset  $S$  of  $V$  of size  $|S| = k$  :  
3     if  $S$  is a vertex cover of  $G$  then  
4       return  $S$   
5   return no
```

What is the time complexity of this algorithm?

- ▶ There are $\binom{n}{k}$ subsets to try.
- ▶ Each subset can be tested in $O(m)$ time.

Remark: m is number of edges. For Vertex Cover we can assume that $m \geq n$

- ▶ $\Rightarrow$ total time $O\left(\binom{n}{k}m\right)$

Test yourself: how large is $\binom{n}{k}$?

(i) $\approx n \cdot k$ (ii) $\approx n^k$ (iii) $\approx k^n$ (iv) $\approx 2^n$ (v) $\approx 2^k n$

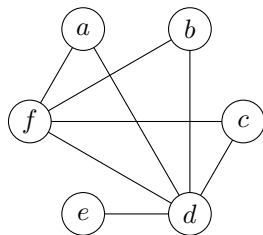
A recursive algorithm

```
1 function VCRec( $G, k, S$ ):  
2   if  $|S| > k$  then return no  
3   if  $S$  is a vertex cover of  $G$  then return  $S$   
4   Let  $(u, v)$  be an edge not covered by  $S$   
5    $X \leftarrow \text{VCRec}(G, k, S \cup \{u\})$   
6   if  $X \neq \text{no}$  then return  $X$   
7   return  $\text{VCRec}(G, k, S \cup \{v\})$ 
```

A recursive algorithm

```
1 function VCRec( $G, k, S$ ):  
2   if  $|S| > k$  then return no  
3   if  $S$  is a vertex cover of  $G$  then return  $S$   
4   Let  $(u, v)$  be an edge not covered by  $S$   
5    $X \leftarrow \text{VCRec}(G, k, S \cup \{u\})$   
6   if  $X \neq \text{no}$  then return  $X$   
7   return  $\text{VCRec}(G, k, S \cup \{v\})$ 
```

Example run (drawn during the lecture)



$$k = 2$$

Analysis overview

We need to analyze both the running time and the correctness of the algorithm.

Do you see how to proceed with these?

Correctness analysis

Intuition: the recursive call $\text{VCRec}(G, k, S)$ tries to find a vertex cover of size $\leq k$ that uses all the vertices in S .

Correctness analysis

Intuition: the recursive call $\text{VCRec}(G, k, S)$ tries to find a vertex cover of size $\leq k$ that uses all the vertices in S .

How should we formalize this property?

Correctness analysis

Intuition: the recursive call $\text{VCRec}(G, k, S)$ tries to find a vertex cover of size $\leq k$ that uses all the vertices in S .

How should we formalize this property?

Suggestion:

Lemma 1.

On input G, k, S , the algorithm returns:

- 1. If there exists a vertex cover T of G such that $S \subseteq T$ and $|T| \leq k$ then the algorithm returns T*
- 2. If no such vertex cover exists then the algorithm returns “no”*

Correctness analysis

Intuition: the recursive call $\text{VCRec}(G, k, S)$ tries to find a vertex cover of size $\leq k$ that uses all the vertices in S .

How should we formalize this property?

Suggestion:

Lemma 1.

On input G, k, S , the algorithm returns:

- 1. If there exists a vertex cover T of G such that $S \subseteq T$ and $|T| \leq k$ then the algorithm returns T*
- 2. If no such vertex cover exists then the algorithm returns “no”*

This claim is false. Do you see why? Can you find a counter-example?

Correctness analysis

Intuition: the recursive call $\text{VCRec}(G, k, S)$ tries to find a vertex cover of size $\leq k$ that uses all the vertices in S .

How should we formalize this property?

New suggestion:

Lemma 2.

On input G, k, S , the algorithm returns either a set T of vertices, or “no”.

- 1. If the algorithm returns a set T then T is a vertex cover such that $S \subseteq T$ and $|T| \leq k$.*
- 2. If the algorithm returns no then no such vertex cover exists.*

Correctness analysis

Intuition: the recursive call $\text{VCRec}(G, k, S)$ tries to find a vertex cover of size $\leq k$ that uses all the vertices in S .

How should we formalize this property?

New suggestion:

Lemma 2.

On input G, k, S , the algorithm returns either a set T of vertices, or “no”.

- 1. If the algorithm returns a set T then T is a vertex cover such that $S \subseteq T$ and $|T| \leq k$.*
- 2. If the algorithm returns no then no such vertex cover exists.*

This lemma is correct. Do you see how to prove it? **Take some time to discuss**

Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.

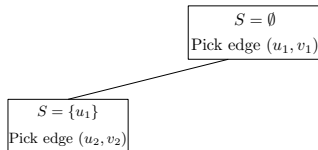
Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.

$S = \emptyset$ Pick edge (u_1, v_1)

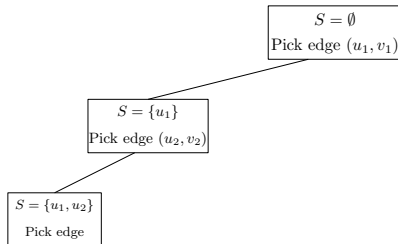
Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



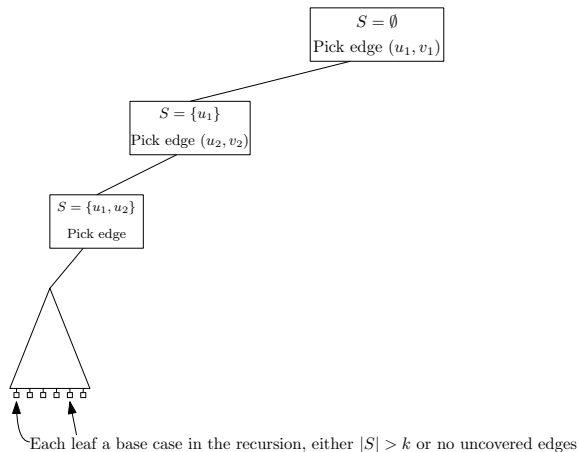
Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



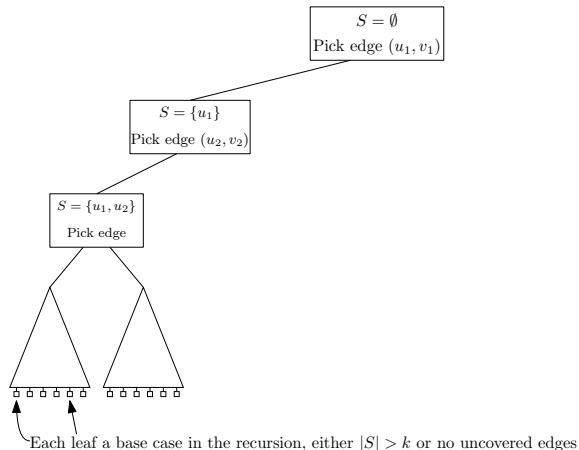
Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



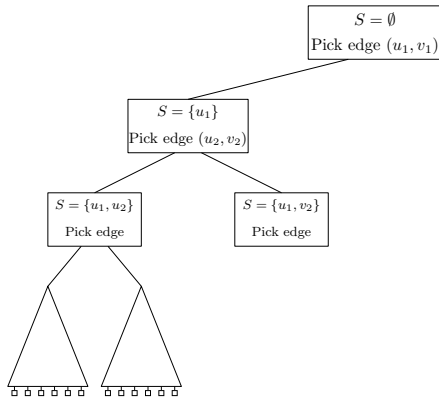
Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



Running time analysis

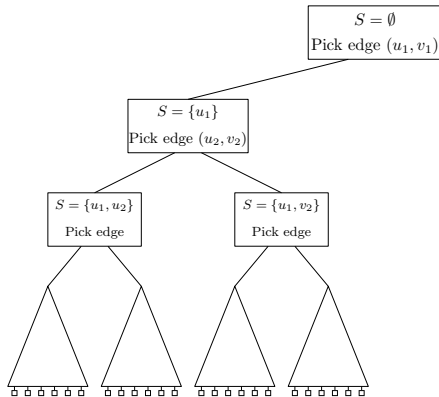
General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



Each leaf a base case in the recursion, either $|S| > k$ or no uncovered edges

Running time analysis

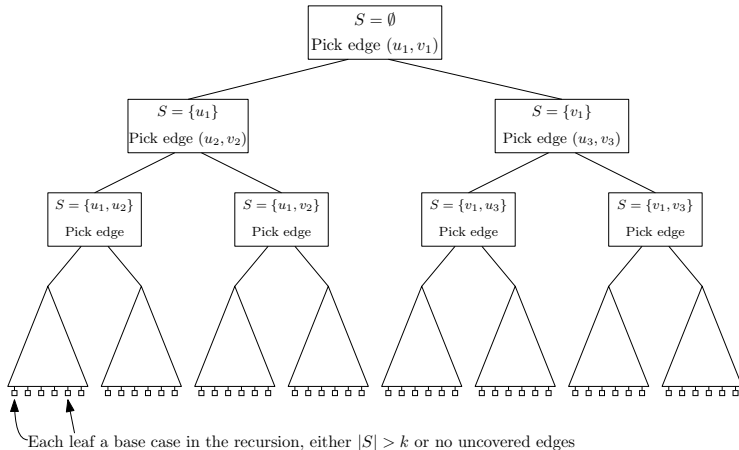
General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



Each leaf a base case in the recursion, either $|S| > k$ or no uncovered edges

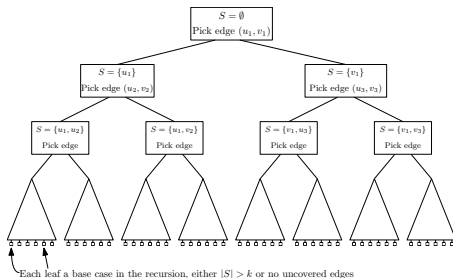
Running time analysis

General strategy for analyzing recursive search algorithms like this:
look at the recursion tree and try to understand it.



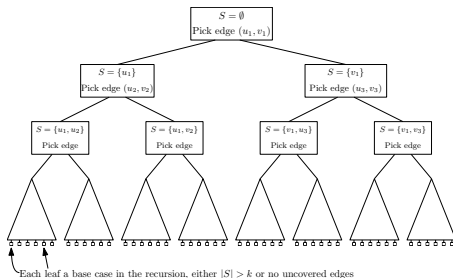
Running time analysis (cont.)

```
1 function VCRec( $G, k, S$ ):  
2   if  $|S| > k$  then return no  
3   if  $S$  is a vertex cover of  $G$  then  
4     return  $S$   
5   Let  $(u, v)$  be an edge not  
6     covered by  $S$   
7    $X \leftarrow \text{VCRec}(G, k, S \cup \{u\})$   
   if  $X \neq \text{no}$  then return  $X$   
   return  $\text{VCRec}(G, k, S \cup \{v\})$ 
```



Running time analysis (cont.)

```
1 function VCRec(G, k, S):  
2   if |S| > k then return no  
3   if S is a vertex cover of G then  
4     return S  
5   Let (u, v) be an edge not  
6   covered by S  
7   X ← VCRec(G, k, S ∪ {u})  
   if X ≠ no then return X  
   return VCRec(G, k, S ∪ {v})
```

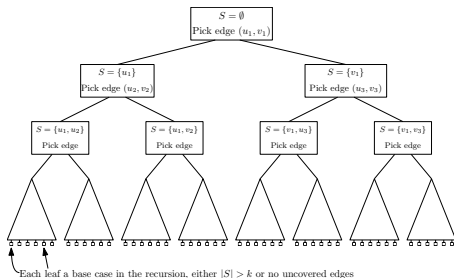


Suppose size of the search tree is σ nodes.

What is then the time complexity?

Running time analysis (cont.)

```
1 function VCRec( $G, k, S$ ):  
2   if  $|S| > k$  then return no  
3   if  $S$  is a vertex cover of  $G$  then  
4     return  $S$   
5   Let  $(u, v)$  be an edge not  
6     covered by  $S$   
7    $X \leftarrow \text{VCRec}(G, k, S \cup \{u\})$   
   if  $X \neq \text{no}$  then return  $X$   
   return  $\text{VCRec}(G, k, S \cup \{v\})$ 
```



Suppose size of the search tree is σ nodes.

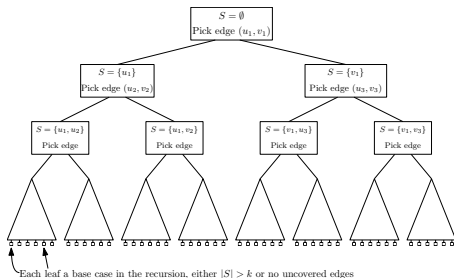
What is then the time complexity?

How much time is needed within each recursive call?

- ▶ Checking if S is a vertex cover: $O(m)$ time
- ▶ Finding an uncovered edge: $O(m)$ time

Running time analysis (cont.)

```
1 function VCRec( $G, k, S$ ):  
2   if  $|S| > k$  then return no  
3   if  $S$  is a vertex cover of  $G$  then  
4     return  $S$   
5   Let  $(u, v)$  be an edge not  
6   covered by  $S$   
7    $X \leftarrow \text{VCRec}(G, k, S \cup \{u\})$   
   if  $X \neq \text{no}$  then return  $X$   
   return  $\text{VCRec}(G, k, S \cup \{v\})$ 
```



Suppose size of the search tree is σ nodes.

What is then the time complexity?

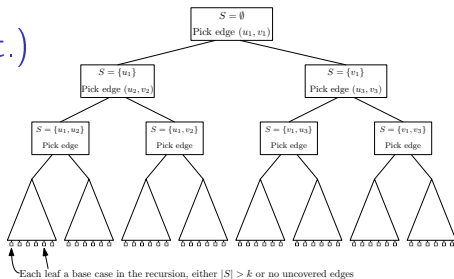
How much time is needed within each recursive call?

- ▶ Checking if S is a vertex cover: $O(m)$ time
- ▶ Finding an uncovered edge: $O(m)$ time

Total running time is $O(\sigma \cdot m)$.

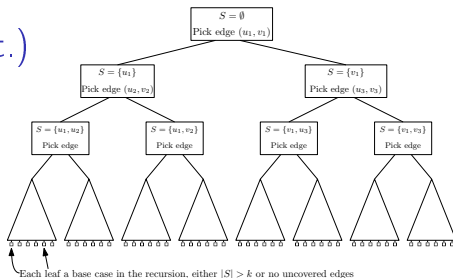
Running time analysis (cont.)

What is the size σ of the search tree, can we analyze it in terms of $k/n/m$?



Running time analysis (cont.)

What is the size σ of the search tree, can we analyze it in terms of $k/n/m$?

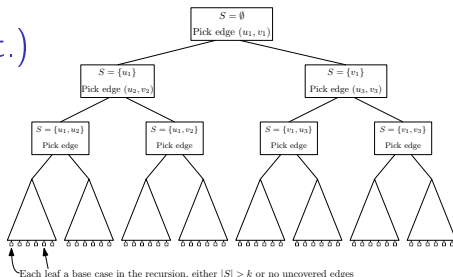


Lemma 3.

The search tree has at most $2^{k+2} = O(2^k)$ nodes.

Running time analysis (cont.)

What is the size σ of the search tree, can we analyze it in terms of $k/n/m$?



Lemma 3.

The search tree has at most $2^{k+2} = O(2^k)$ nodes.

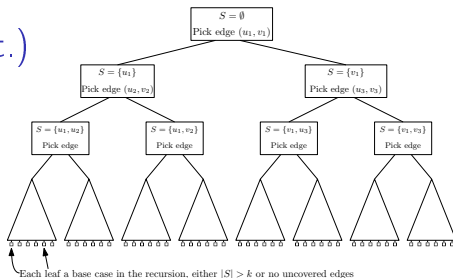
Proof outline.

1. Prove that the search tree has depth at most $k + 1$ (depth = max number of edges on the path from the root to leaf).



Running time analysis (cont.)

What is the size σ of the search tree, can we analyze it in terms of $k/n/m$?



Lemma 3.

The search tree has at most $2^{k+2} = O(2^k)$ nodes.

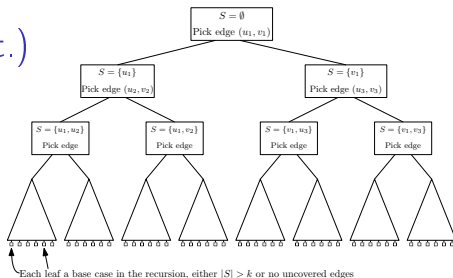
Proof outline.

1. Prove that the search tree has depth at most $k + 1$ (depth = max number of edges on the path from the root to leaf).
2. In general, a binary tree of depth d has at most 2^d leaf nodes so there are at most 2^{k+1} leaves.



Running time analysis (cont.)

What is the size σ of the search tree, can we analyze it in terms of $k/n/m$?



Lemma 3.

The search tree has at most $2^{k+2} = O(2^k)$ nodes.

Proof outline.

1. Prove that the search tree has depth at most $k + 1$ (depth = max number of edges on the path from the root to leaf).
2. In general, a binary tree of depth d has at most 2^d leaf nodes so there are at most 2^{k+1} leaves.
3. A tree with L leaves has at most $2L - 1$ nodes, so there are at most $2 \cdot 2^{k+1} - 1 < 2^{k+2}$ nodes in our search tree.



Agenda

Vertex Cover

Parameterized Complexity

Kernelization

Closest String

Parameterized Problems

A **parameterized problem** is a problem where the input consists of some data x and a numeric parameter k .

Parameterized Problems

A **parameterized problem** is a problem where the input consists of some data x and a numeric parameter k .

Examples:

- ▶ Given graph G and parameter k , is there a vertex cover of size $\leq k$?
- ▶ Given a Boolean logical formula and parameter k , is there a satisfying assignment that sets $\leq k$ variables to true?

Parameterized Problems

A **parameterized problem** is a problem where the input consists of some data x and a numeric parameter k .

Examples:

- ▶ Given graph G and parameter k , is there a vertex cover of size $\leq k$?
- ▶ Given a Boolean logical formula and parameter k , is there a satisfying assignment that sets $\leq k$ variables to true?

Underlying idea:

We want parameter to somehow “explain” the complexity of the problem

If k is small the problem should be efficiently solvable.

Choice of parameter

Many possible choices for what to use as parameter for any given problem.

Choice of parameter

Many possible choices for what to use as parameter for any given problem.

E.g. some possible different parameterized variants of Max-Cut:

Choice of parameter

Many possible choices for what to use as parameter for any given problem.

E.g. some possible different parameterized variants of Max-Cut:

- ▶ Given graph G on m edges and parameter k , is there a cut which cuts $\geq k$ edges?

Choice of parameter

Many possible choices for what to use as parameter for any given problem.

E.g. some possible different parameterized variants of Max-Cut:

- ▶ Given graph G on m edges and parameter k , is there a cut which cuts $\geq k$ edges?
- ▶ Given graph G on m edges and parameter k , is there a cut which cuts $\geq m - k$ edges?

Choice of parameter

Many possible choices for what to use as parameter for any given problem.

E.g. some possible different parameterized variants of Max-Cut:

- ▶ Given graph G on m edges and parameter k , is there a cut which cuts $\geq k$ edges?
- ▶ Given graph G on m edges and parameter k , is there a cut which cuts $\geq m - k$ edges?
- ▶ Given graph G and parameter k , what is max cut if one side can only have $\leq k$ vertices?

Key Concept: Fixed-Parameter Tractability

Definition 4.

A problem is **fixed-parameter tractable** (FPT) with respect to a parameter k if it has an algorithm running in time $f(k) \cdot n^{O(1)}$, where n is the problem size.

Key Concept: Fixed-Parameter Tractability

Definition 4.

A problem is **fixed-parameter tractable** (FPT) with respect to a parameter k if it has an algorithm running in time $f(k) \cdot n^{O(1)}$, where n is the problem size.

Note: f is a function of k only, does not depend on n

Key Concept: Fixed-Parameter Tractability

Definition 4.

A problem is **fixed-parameter tractable** (FPT) with respect to a parameter k if it has an algorithm running in time $f(k) \cdot n^{O(1)}$, where n is the problem size.

Note: f is a function of k only, does not depend on n

Example: the $O(2^k \cdot m)$ algorithm for Vertex Cover shows that Vertex Cover is FPT with respect to the vertex cover size.

Key Concept: Fixed-Parameter Tractability

Definition 4.

A problem is **fixed-parameter tractable** (FPT) with respect to a parameter k if it has an algorithm running in time $f(k) \cdot n^{O(1)}$, where n is the problem size.

Note: f is a function of k only, does not depend on n

Example: the $O(2^k \cdot m)$ algorithm for Vertex Cover shows that Vertex Cover is FPT with respect to the vertex cover size.

Test yourself: which of these time complexities establish that a problem is FPT?

- (i) $O(n^{1000})$ (ii) $O(n^{\log \log k})$ (iii) $O(k \cdot 2^n)$
(iv) $O(2^{2^{2^k}} \cdot n^{100})$ (v) $O((\log n)^{\log k} \cdot n^3)$

Key Concept: Fixed-Parameter Tractability

Definition 4.

A problem is **fixed-parameter tractable** (FPT) with respect to a parameter k if it has an algorithm running in time $f(k) \cdot n^{O(1)}$, where n is the problem size.

Note: f is a function of k only, does not depend on n

Example: the $O(2^k \cdot m)$ algorithm for Vertex Cover shows that Vertex Cover is FPT with respect to the vertex cover size.

Test yourself: which of these time complexities establish that a problem is FPT?

- (i) $O(n^{1000})$
- (ii) $O(n^{\log \log k})$
- (iii) $O(k \cdot 2^n)$
- (iv) $O(2^{2^k} \cdot n^{100})$
- (v) $O((\log n)^{\log k} \cdot n^3)$

Agenda

Vertex Cover

Parameterized Complexity

Kernelization

Closest String

Vertex Cover again

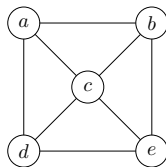
Let us look at another FPT algorithm for Vertex Cover.

Vertex Cover again

Let us look at another FPT algorithm for Vertex Cover.

Motivating example:

if $\text{Opt} \leq 3$ then node c *must* be used in every optimal solution.



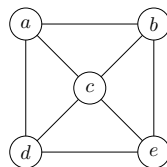
$$k = 3$$

Vertex Cover again

Let us look at another FPT algorithm for Vertex Cover.

Motivating example:

if $\text{Opt} \leq 3$ then node c *must* be used in every optimal solution.



$k = 3$

More generally:

Lemma 5.

Any vertex cover of size $\leq k$ must include all vertices with degree $\geq k + 1$.

Vertex Cover degree heuristic

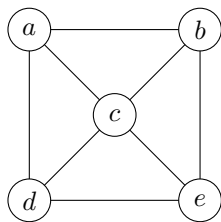
Suggests the following procedure:

1. Remove all vertices of degree $> k$ (put them in vertex cover)
2. Solve the remaining graph (somehow).

Vertex Cover degree heuristic

Suggests the following procedure:

1. Remove all vertices of degree $> k$ (put them in vertex cover)
2. Solve the remaining graph (somehow).

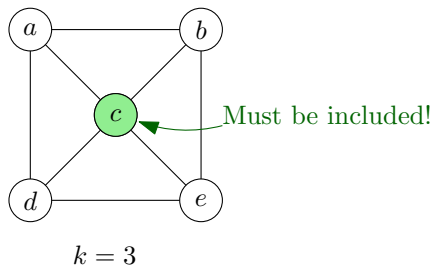


$$k = 3$$

Vertex Cover degree heuristic

Suggests the following procedure:

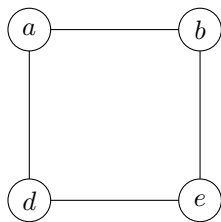
1. Remove all vertices of degree $> k$ (put them in vertex cover)
2. Solve the remaining graph (somehow).



Vertex Cover degree heuristic

Suggests the following procedure:

1. Remove all vertices of degree $> k$ (put them in vertex cover)
2. Solve the remaining graph (somehow).

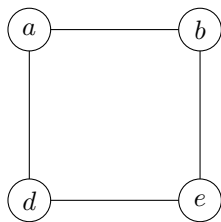


$$k = \cancel{3} 2$$

Vertex Cover degree heuristic

Suggests the following procedure:

1. Remove all vertices of degree $> k$ (put them in vertex cover)
2. Solve the remaining graph (somehow).



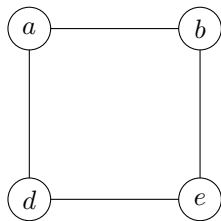
$$k = 2$$

How big can the remaining graph be?

Vertex Cover degree heuristic

Suggests the following procedure:

1. Remove all vertices of degree $> k$ (put them in vertex cover)
2. Solve the remaining graph (somehow).



$$k = \cancel{3} 2$$

How big can the remaining graph be?

Problem(?): maybe all vertices had small degree to start with and we just have some big graph with low degrees?

Understanding the heuristic

Suppose we have a graph where all vertices have degree $\leq d$ and there is a vertex cover of size k .

How big can such a graph be?

(Try with $d = 3$ and $k = 2$. Can you make such a graph with 10 edges?)

Understanding the heuristic

Suppose we have a graph where all vertices have degree $\leq d$ and there is a vertex cover of size k .

How big can such a graph be?

(Try with $d = 3$ and $k = 2$. Can you make such a graph with 10 edges?)

Lemma 6.

If maximum degree in G is $\leq d$ and G has a vertex cover of size $\leq k$ then G has at most $k \cdot d$ edges.

Understanding the heuristic

Suppose we have a graph where all vertices have degree $\leq d$ and there is a vertex cover of size k .

How big can such a graph be?

(Try with $d = 3$ and $k = 2$. Can you make such a graph with 10 edges?)

Lemma 6.

If maximum degree in G is $\leq d$ and G has a vertex cover of size $\leq k$ then G has at most $k \cdot d$ edges.

Proof.

Each of the k vertices used in the minimum vertex cover can touch at most d different edges. But in total the vertex cover touches all edges. □

Algorithm

```
1 function VertexCover( $G, k$ ):  
2   Let  $S_1$  be set of nodes of degree  $> k$   
3   Let  $G'$  be graph obtained by removing  $S_1$  and isolated vertices  
   from  $G$   
4   if  $|E(G')| > k^2$  then return no  
5   Let  $S_2$  be minimum vertex cover in  $G'$   
6   if  $|S_1| + |S_2| > k$  then return no  
7   return  $S_1 \cup S_2$ 
```

Algorithm

```
1 function VertexCover( $G, k$ ):
2   Let  $S_1$  be set of nodes of degree  $> k$ 
3   Let  $G'$  be graph obtained by removing  $S_1$  and isolated vertices
   from  $G$ 
4   if  $|E(G')| > k^2$  then return no
5   Let  $S_2$  be minimum vertex cover in  $G'$ 
6   if  $|S_1| + |S_2| > k$  then return no
7   return  $S_1 \cup S_2$ 
```

Correctness of algorithm can be proved using the previous lemmas.

Algorithm

```
1 function VertexCover( $G, k$ ):  
2   Let  $S_1$  be set of nodes of degree  $> k$   
3   Let  $G'$  be graph obtained by removing  $S_1$  and isolated vertices  
   from  $G$   
4   if  $|E(G')| > k^2$  then return no  
5   Let  $S_2$  be minimum vertex cover in  $G'$   
6   if  $|S_1| + |S_2| > k$  then return no  
7   return  $S_1 \cup S_2$ 
```

Correctness of algorithm can be proved using the previous lemmas.

Time complexity analysis:

- ▶ Computing S_1 and G' can be done in $O(n + m)$ time.
- ▶ How to find S_2 ? Depends on algorithm used:
 - ▶ Naive brute force: $O(2^{|V(G')|}) = O(2^{k^2})$
 - ▶ Use previous FPT algorithm: $O(2^k \cdot k^2)$
 - ▶ Either way, this running time depends only on k
- ▶ Total running time $O(n + m + 2^k k^2)$ (using previous FPT alg).

Kernelization

What did we do?

Kernelization

What did we do?

We saw that any instance of Vertex Cover can be “compressed” to an equivalent instance of size depending only on k .

Kernelization

What did we do?

We saw that any instance of Vertex Cover can be “compressed” to an equivalent instance of size depending only on k .

This is called **kernelization**.

Definition 7.

Consider a parameterized *decision* problem (answer is YES/NO). A **kernelization algorithm** is a polynomial time algorithm that takes as input instance (x, k) and outputs instance (x', k') such that:

1. (x, k) is a YES-instance if and only if (x', k') is a YES-instance.
2. there is a computable function f such that $|x'| \leq f(k)$ and $k' \leq k$

Kernelization

Definition 7.

Consider a parameterized *decision* problem (answer is YES/NO). A **kernelization algorithm** is a polynomial time algorithm that takes as input instance (x, k) and outputs instance (x', k') such that:

1. (x, k) is a YES-instance if and only if (x', k') is a YES-instance.
2. there is a computable function f such that $|x'| \leq f(k)$ and $k' \leq k$

The instance (x', k') is called a **kernel**.

Kernelization

Definition 7.

Consider a parameterized *decision* problem (answer is YES/NO). A **kernelization algorithm** is a polynomial time algorithm that takes as input instance (x, k) and outputs instance (x', k') such that:

1. (x, k) is a YES-instance if and only if (x', k') is a YES-instance.
2. there is a computable function f such that $|x'| \leq f(k)$ and $k' \leq k$

The instance (x', k') is called a **kernel**.

We showed that Vertex Cover has a kernel of size $\leq k^2$.

Kernelization vs FPT

Kernelization and FPT are intimately related.

Theorem 8.

A parameterized problem is FPT if and only if it is decidable and has a kernelization algorithm.

Kernelization vs FPT

Kernelization and FPT are intimately related.

Theorem 8.

A parameterized problem is FPT if and only if it is decidable and has a kernelization algorithm.

Proof sketch.

In one direction, if a problem has a kernelization algorithm then we can construct an FPT algorithm:

Run kernelization algorithm in time $\text{poly}(n)$ to get a small instance of size $f(k)$, then run any algorithm A on the small instance. If A has running time $O(T(n))$ this gives an algorithm running in time $O(\text{poly}(n) + T(f(k)))$.



Kernelization vs FPT

Kernelization and FPT are intimately related.

Theorem 8.

A parameterized problem is FPT if and only if it is decidable and has a kernelization algorithm.

Proof sketch.

In the other direction, suppose there is an FPT algorithm running in time $O(f(k) \cdot n^c)$.

Given an input (x, k) , two cases:



Kernelization vs FPT

Kernelization and FPT are intimately related.

Theorem 8.

A parameterized problem is FPT if and only if it is decidable and has a kernelization algorithm.

Proof sketch.

In the other direction, suppose there is an FPT algorithm running in time $O(f(k) \cdot n^c)$.

Given an input (x, k) , two cases:

1. if $n \leq f(k)$ then output (x, k) itself as the kernel.



Kernelization vs FPT

Kernelization and FPT are intimately related.

Theorem 8.

A parameterized problem is FPT if and only if it is decidable and has a kernelization algorithm.

Proof sketch.

In the other direction, suppose there is an FPT algorithm running in time $O(f(k) \cdot n^c)$.

Given an input (x, k) , two cases:

1. if $n \leq f(k)$ then output (x, k) itself as the kernel.
2. if $n > f(k)$ then run the FPT algorithm to solve the input in time $O(f(k)n^c) = O(n^{c+1})$ and return a trivial yes/no instance.



Agenda

Vertex Cover

Parameterized Complexity

Kernelization

Closest String

The Closest String problem

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ

Output: a string y of length ℓ

Goal: minimize the maximum difference $\max_{i=1..n} \Delta(x_i, y)$

The Closest String problem

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ

Output: a string y of length ℓ

Goal: minimize the maximum difference $\max_{i=1..n} \Delta(x_i, y)$

For two strings x and y , $\Delta(x, y)$ denotes their Hamming distance
(number of positions where they differ)

► E.g. $\Delta(\text{str}\text{a}\text{n}\text{g}\text{e}\text{r}\text{s}, \text{str}\text{o}\text{n}\text{g}\text{e}\text{s}\text{t}) = 3$

The Closest String problem

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ

Output: a string y of length ℓ

Goal: minimize the maximum difference $\max_{i=1..n} \Delta(x_i, y)$

For two strings x and y , $\Delta(x, y)$ denotes their Hamming distance (number of positions where they differ)

► E.g. $\Delta(\text{str}\text{a}\text{ng}\text{er}\text{s}, \text{str}\text{ong}\text{est}) = 3$

For example think of $x_1, \dots, x_n$ being n different DNA sequences. The problem asks for a new DNA sequence y which is close to all of them.

The Closest String problem

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ

Output: a string y of length ℓ

Goal: minimize the maximum difference $\max_{i=1..n} \Delta(x_i, y)$

For two strings x and y , $\Delta(x, y)$ denotes their Hamming distance (number of positions where they differ)

► E.g. $\Delta(\text{str}\text{a}\text{n}\text{g}\text{e}\text{r}\text{s}, \text{str}\text{o}\text{n}\text{g}\text{e}\text{s}\text{t}) = 3$

For example think of $x_1, \dots, x_n$ being n different DNA sequences. The problem asks for a new DNA sequence y which is close to all of them.

Poll: have you seen this problem before?

Closest String parameters

What should we choose as parameter to make Closest String a parameterized problem?

Closest String parameters

What should we choose as parameter to make Closest String a parameterized problem?

At least two natural choices:

- ▶ Parameterize by n , the number of input strings.
- ▶ Parameterize by **Opt**, the optimum value.

Closest String parameters

What should we choose as parameter to make Closest String a parameterized problem?

At least two natural choices:

- ▶ Parameterize by n , the number of input strings.
- ▶ Parameterize by **Opt**, the optimum value.

The problem turns out to be FPT with respect to both parameters.

Closest String parameters

What should we choose as parameter to make Closest String a parameterized problem?

At least two natural choices:

- ▶ Parameterize by n , the number of input strings.
- ▶ Parameterize by **Opt**, the optimum value.

The problem turns out to be FPT with respect to both parameters.

We will look at an algorithm for the second option.

Closest String parameterized by d

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ and a parameter d

Output: a string y of length ℓ such that

$\max_{i=1..n} \Delta(x_i, y) \leq d$, or “no” if no such y exists.

Closest String parameterized by d

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ and a parameter d

Output: a string y of length ℓ such that

$\max_{i=1..n} \Delta(x_i, y) \leq d$, or “no” if no such y exists.

Our goal: find an algorithm running in time $f(d) \cdot (n \cdot \ell)^{O(1)}$ for some function f depending only on d .

Closest String parameterized by d

Input: n strings $x_1, x_2, \dots, x_n$ of length ℓ and a parameter d

Output: a string y of length ℓ such that

$\max_{i=1..n} \Delta(x_i, y) \leq d$, or “no” if no such y exists.

Our goal: find an algorithm running in time $f(d) \cdot (n \cdot \ell)^{O(1)}$ for some function f depending only on d .

Ideas for how to get started?

Idea 1: Local Search

Let us try to do some kind of local search:

- ▶ At every step, we are at some string y

Idea 1: Local Search

Let us try to do some kind of local search:

- ▶ At every step, we are at some string y
- ▶ If y already a solution, great!

Idea 1: Local Search

Let us try to do some kind of local search:

- ▶ At every step, we are at some string y
- ▶ If y already a solution, great!
- ▶ Otherwise recursively try different ways of changing a character in y to improve it.

Idea 1: Local Search

Let us try to do some kind of local search:

- ▶ At every step, we are at some string y
- ▶ If y already a solution, great!
- ▶ Otherwise recursively try different ways of changing a character in y to improve it.

Problem(s):

- ▶ How do we know when to give up and stop searching?

Idea 1: Local Search

Let us try to do some kind of local search:

- ▶ At every step, we are at some string y
- ▶ If y already a solution, great!
- ▶ Otherwise recursively try different ways of changing a character in y to improve it.

Problem(s):

- ▶ How do we know when to give up and stop searching?
- ▶ How do we prevent oscillating between two states?
E.g. if $x_1 = AAA$, $x_2 = BBB$ and $k = 1$ need to avoid oscillating between $y = ABB$ and $y = AAB$.

Idea 2: Bounded Depth Search

Looking back at the first Vertex Cover algorithm, a crucial property that made it work was that the search could never go deeper than depth $k + 1$.

Idea 2: Bounded Depth Search

Looking back at the first Vertex Cover algorithm, a crucial property that made it work was that the search could never go deeper than depth $k + 1$.

Could we achieve something similar here?

Idea 2: Bounded Depth Search

Looking back at the first Vertex Cover algorithm, a crucial property that made it work was that the search could never go deeper than depth $k + 1$.

Could we achieve something similar here?

Solution: start our local search at x_1 .

Then we should never search deeper than depth d .

Closest String Algorithm

```
1 function ClosestString( $y, s, x_1, \dots, x_n, d$ ):  
2   if  $s > d$  then return no  
3   if  $\max_i \Delta(x_i, y) \leq d$  then return  $y$   
4   Let  $i$  be such that  $\Delta(x_i, y) > d$   
5   Let  $J$  be set of first  $d + 1$  indices where  $x_i$  differs from  $y$   
6   foreach  $j \in J$  :  
7      $y' \leftarrow y$   
8      $y'[j] \leftarrow x_i[j]$   
9      $z \leftarrow \text{ClosestString}(y', s + 1, x_1, \dots, x_n, d)$   
10    if  $z \neq \text{no}$  then return  $z$   
11  return no
```

Closest String Algorithm

```
1 function ClosestString( $y, s, x_1, \dots, x_n, d$ ):  
2   if  $s > d$  then return no  
3   if  $\max_i \Delta(x_i, y) \leq d$  then return  $y$   
4   Let  $i$  be such that  $\Delta(x_i, y) > d$   
5   Let  $J$  be set of first  $d + 1$  indices where  $x_i$  differs from  $y$   
6   foreach  $j \in J$  :  
7      $y' \leftarrow y$   
8      $y'[j] \leftarrow x_i[j]$   
9      $z \leftarrow \text{ClosestString}(y', s + 1, x_1, \dots, x_n, d)$   
10    if  $z \neq \text{no}$  then return  $z$   
11  return no
```

Given an input $x_1, \dots, x_n$ and parameter d , we start the search by calling $\text{ClosestString}(x_1, 0, x_1, \dots, x_n, d)$.

Correctness of algorithm

Two things to prove.

Lemma 9.

If algorithm returns a string y then it is a solution.

Lemma 10.

If a solution exists then algorithm finds a solution.

Correctness of algorithm

Two things to prove.

Lemma 9.

If algorithm returns a string y then it is a solution.

Lemma 10.

If a solution exists then algorithm finds a solution.

Lemma 9 should be easy to prove!

Correctness of algorithm

Two things to prove.

Lemma 9.

If algorithm returns a string y then it is a solution.

Lemma 10.

If a solution exists then algorithm finds a solution.

Lemma 9 should be easy to prove!

Lemma 10 may be more challenging. As always with recursive algorithms, the most natural strategy is to use induction. Very similar to correctness proof of the first Vertex Cover algorithm.

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)
2. If the recursion tree has size σ we get running time $O(\sigma \cdot \ell \cdot n)$.

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)
2. If the recursion tree has size σ we get running time $O(\sigma \cdot \ell \cdot n)$.

How large can the size σ of recursion tree be?

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)
2. If the recursion tree has size σ we get running time $O(\sigma \cdot \ell \cdot n)$.

How large can the size σ of recursion tree be?

We have a very similar situation as in Vertex Cover:

- Search tree has depth at most $d + 1$.

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)
2. If the recursion tree has size σ we get running time $O(\sigma \cdot \ell \cdot n)$.

How large can the size σ of recursion tree be?

We have a very similar situation as in Vertex Cover:

- ▶ Search tree has depth at most $d + 1$.
- ▶ Instead of a binary tree, it is a $(d + 1)$ -ary tree (each node has $d + 1$ children)

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)
2. If the recursion tree has size σ we get running time $O(\sigma \cdot \ell \cdot n)$.

How large can the size σ of recursion tree be?

We have a very similar situation as in Vertex Cover:

- ▶ Search tree has depth at most $d + 1$.
- ▶ Instead of a binary tree, it is a $(d + 1)$ -ary tree (each node has $d + 1$ children)
- ▶ So $\sigma = O((d + 1)^{d+1}) = O(d^{d+1})$

Time complexity of algorithm

Analyze using same strategy as the Vertex Cover algorithm:

1. Each individual call of the recursion can be done in $O(\ell \cdot n)$ time. (convince yourself of this!)
2. If the recursion tree has size σ we get running time $O(\sigma \cdot \ell \cdot n)$.

How large can the size σ of recursion tree be?

We have a very similar situation as in Vertex Cover:

- ▶ Search tree has depth at most $d + 1$.
- ▶ Instead of a binary tree, it is a $(d + 1)$ -ary tree (each node has $d + 1$ children)
- ▶ So $\sigma = O((d + 1)^{d+1}) = O(d^{d+1})$

Putting this together, running time is $O(d^{d+1} \cdot \ell \cdot n)$.

FPT in the parameter d as promised!

Thank you!